Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине «АПО ЭВМиС»
Тема: «Изучение аппаратно-программной архитектуры процессоров семейства Intel – разработка ассемблерной вставки».

Выполнил:
Студент 4-го курса
Группы АС-66
Невар В.А.
Проверила:
Кулик

Брест 2026

Цель работы: изучить программную модель микропроцессора, регистры и организацию памяти; получить практические навыки работы с ассемблерными вставками.

Вариант №14

Задание: удалить повторяющиеся пробелы, а также пробелы в начале и в конце строки. Обработку строки оформить в виде отдельной функции с ассемблерной вставкой, реализовать ту же обработку функцией на C++ и сравнить быстродействие обеих реализаций. Среда разработки – MS Visual Studio (платформа x86).

Ход работы

Алгоритм. Обработка выполняется на месте двумя указателями: из первого происходит чтение символов, во второй – запись результата. Сначала пропускаются пробелы в начале строки. Далее символы читаются до нулевого терминатора: любой символ, кроме пробела, копируется; пробел копируется только в том случае, если последний записанный символ не является пробелом. Если после обработки последним записанным символом оказался пробел, он удаляется, после чего записывается нулевой терминатор.

Ассемблерная вставка. Адрес строки загружается в регистры esi (чтение) и edi (запись), в регистре ebx хранится адрес начала строки. Для чтения и записи используются цепочечные команды lodsb и stosb, флаг направления сбрасывается командой cld. Условие окончания – символ с кодом 0, пробел – код 20h.

Проверка и сравнение. Программа запрашивает строку, выводит исходную строку и результаты обеих функций (в квадратных скобках, чтобы были видны пробелы), затем замеряет время работы функций на строке длиной 20 000 000 символов (среднее значение по 10 запусков).

Текст программы:

```
#include <iostream>
#include <vector>
#include <random>
#include <chrono>
#include <cstring>
#include <windows.h>
using namespace std;
using StringFunc = void (*)(char*);
const int MAX_LEN = 256;
const size_t BENCH_LEN = 20000000;
const int BENCH_RUNS = 10;
#if defined(_M_X64)
static StringFunc load_asm()
{
static const unsigned char code[] =
{
0x48, 0x89, 0xCA, 0x49, 0x89, 0xC8, 0x80, 0x3A, 0x20, 0x75, 0x05, 0x48,
0xFF, 0xC2, 0xEB, 0xF6, 0x8A, 0x02, 0x48, 0xFF, 0xC2, 0x84, 0xC0, 0x74,
0x13, 0x3C, 0x20, 0x75, 0x07, 0x41, 0x80, 0x78, 0xFF, 0x20, 0x74, 0xEC,
0x41, 0x88, 0x00, 0x49, 0xFF, 0xC0, 0xEB, 0xE4, 0x49, 0x39, 0xC8, 0x74,
0x0A, 0x41, 0x80, 0x78, 0xFF, 0x20, 0x75, 0x03, 0x49, 0xFF, 0xC8, 0x41,
```

```
0xC6, 0x00, 0x00, 0xC3
};
void* mem = VirtualAlloc(nullptr, sizeof(code), MEM_COMMIT | MEM_RESERVE,
PAGE_EXECUTE_READWRITE);
memcpy(mem, code, sizeof(code));
FlushInstructionCache(GetCurrentProcess(), mem, sizeof(code));
return reinterpret_cast<StringFunc>(mem);
}
static const StringFunc asm_impl = load_asm();
void remove_spaces_asm(char* str)
{
asm_impl(str);
}
#elif defined(_M_IX86)
void remove_spaces_asm(char* buf)
{
__asm
{
cld
mov esi, buf
mov edi, esi
mov ebx, esi
skip_lead:
lodsb
cmp al, 20h
je skip_lead
dec esi
next_char:
lodsb
test al, al
jz trim_end
cmp al, 20h
jne store_char
cmp byte ptr [edi - 1], 20h
je next_char
store_char:
stosb
jmp next_char
trim_end:
cmp edi, ebx
je put_zero
cmp byte ptr [edi - 1], 20h
jne put_zero
dec edi
put_zero:
mov byte ptr [edi], 0
}
}
#else
#error Only x86 and x64 are supported
#endif
void remove_spaces_cpp(char* str)
{
const char* src = str;
char* dst = str;
while (*src == ' ')
++src;
while (*src != '\0')
{
char c = *src++;
if (c != ' ' || dst[-1] != ' ')
*dst++ = c;
}
if (dst > str && dst[-1] == ' ')
--dst;
*dst = '\0';
}
const char* yes_no(bool value)
{
return value ? "yes" : "no";
```

```
}
void print_string(const char* title, const char* str)
{
cout << title << "[" << str << "] (length: " << strlen(str) << ")" << endl;
}
void fill_test_data(vector<char>& data)
{
mt19937 gen(12345);
uniform_int_distribution<int> percent(0, 99);
uniform_int_distribution<int> letter(0, 25);
for (size_t i = 0; i + 1 < data.size(); ++i)
data[i] = percent(gen) < 40 ? ' ' : static_cast<char>('a' + letter(gen));
for (size_t i = 0; i < 5; ++i)
{
data[i] = ' ';
data[data.size() - 2 - i] = ' ';
}
data[data.size() - 1] = '\0';
}
double measure(StringFunc func, const vector<char>& source,
vector<char>& work)
{
double total = 0.0;
for (int run = 0; run < BENCH_RUNS; ++run)
{
memcpy(work.data(), source.data(), source.size());
auto start = chrono::steady_clock::now();
func(work.data());
auto finish = chrono::steady_clock::now();
total += chrono::duration<double, milli>(finish - start).count();
}
return total / BENCH_RUNS;
}
void run_benchmark()
{
vector<char> source(BENCH_LEN);
vector<char> work_asm(BENCH_LEN);
vector<char> work_cpp(BENCH_LEN);
fill_test_data(source);
cout << endl << "Performance comparison" << endl;
cout << "Test string length: " << BENCH_LEN - 1 << " characters" << endl;
cout << "Number of runs: " << BENCH_RUNS << endl;
double time_asm = measure(remove_spaces_asm, source, work_asm);
double time_cpp = measure(remove_spaces_cpp, source, work_cpp);
bool same = strcmp(work_asm.data(), work_cpp.data()) == 0;
cout << "Function results match: " << yes_no(same) << endl;
cout << "Assembly insert: " << time_asm << " ms" << endl;
cout << "C++ function: " << time_cpp << " ms" << endl;
}
int main()
{
SetConsoleCP(1251);
SetConsoleOutputCP(1251);
cout << "Laboratory work No. 1. Variant 14" << endl;
cout << "Task: remove repeated spaces, as well as spaces" << endl;
cout << "at the beginning and at the end of the string." << endl << endl;
char input[MAX_LEN];
cout << "Enter a string (at most " << MAX_LEN - 1 << " characters):" << endl;
cin.getline(input, MAX_LEN);
if (cin.fail())
{
cin.clear();
cout << "The string is too long, its first part is used." << endl;
}
char text_asm[MAX_LEN];
char text_cpp[MAX_LEN];
memcpy(text_asm, input, MAX_LEN);
memcpy(text_cpp, input, MAX_LEN);
remove_spaces_asm(text_asm);
remove_spaces_cpp(text_cpp);
```

```
cout << endl;
print_string("Original string: ", input);
print_string("Result (asm insert): ", text_asm);
print_string("Result (C++): ", text_cpp);
bool same = strcmp(text_asm, text_cpp) == 0;
cout << "Results match: " << yes_no(same) << endl;
run_benchmark();
return 0;
}
```

Результат:

```
Laboratory work No. 1. Variant 14
Task: remove repeated spaces, as well as spaces
at the beginning and at the end of the string.

Enter a string (at most 255 characters):
ladnoooo

Original string:         [ladnoooo]  (length: 8)
Result (asm insert):     [ladnoooo]  (length: 8)
Result (C++):            [ladnoooo]  (length: 8)
Results match: yes

Performance comparison
Test string length: 19999999 characters
Number of runs: 10
Function results match: yes
Assembly insert: 67.276 ms
C++ function:    99.1862 ms
```

Результат работы программы

Вывод: Я изучил программную модель микропроцессора, регистры и организацию памяти; получил практические навыки работы с ассемблерными вставками.